

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- warum verschiedene Algorithmen für dasselbe Problem unterschiedlich effizient sein können
- was die Landau-Symbole O , Ω und Θ aussagen und wie sie sich voneinander unterscheiden
- wie aus der Struktur eines rekursiven Algorithmus eine Rekurrenzgleichung entsteht

Sie können...

- die Anzahl dominanter Operationen eines Algorithmus in Abhängigkeit von n bestimmen
- einen Algorithmus einer Komplexitätsklasse zuordnen und dies begründen
- das Mastertheorem anwenden

Effizienz von Algorithmen

Effizienz von Algorithmen

Sind alle Algorithmen für eine Problemklasse gleich effizient?



Wir wissen bereits: Algorithmen auch für eine Problemklasse können sich unterscheiden hinsichtlich

- der Anzahl der ausgeführten Befehle (Laufzeitkomplexität)
- der Anzahl der benutzten Speicherzellen (Speicherkomplexität)

Maximale Abschnittssumme

Beispielaufgabenstellung

Es soll die maximale Abschnittssumme für eine Abfolge von n ganzen Zahlen gefunden werden. Die Zahlen liegen in einem Feld Z mit den Indizes $0 \dots n - 1$ im Speicher vor.

Hat man z.B. die Zahlenfolge

-59, 52, 46, 14, -50, 58, -87, -77, 34, 15

liefert folgende Teil-Zahlenfolge die maximale Abschnittssumme:

$$\mathbf{52 + 46 + 14 + -50 + 58 = 120}$$

Es ist offensichtlich, dass die Anzahl der Befehle zur Ermittlung der maximalen Abschnittssumme mit der Länge n der Zahlenfolge zunimmt.

Maximale Abschnittssumme

Erster Lösungsansatz

Ein möglicher Lösungsansatz ist, alle Teilfolgen zu betrachten, deren Summen zu berechnen und das Maximum auszuwählen.

Betrachtet man die Anzahl der Additionen in Zeile 7, so erkennt man, dass deren Anzahl aufgrund der drei verschachtelten Schleifen in etwa proportional zu n^3 zunimmt.

Algorithmen mit einem solchen Zeitverhalten nennt man **kubische Algorithmen**.

MAXFOLGE1(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:   for  $j \leftarrow i$  to  $n - 1$  do
5:      $sum \leftarrow 0$ 
6:     for  $k \leftarrow i$  to  $j$  do
7:        $sum \leftarrow sum + Z[k]$ 
8:     end for
9:     if  $sum > max$  then
10:       $max \leftarrow sum$ 
11:       $links \leftarrow i$ 
12:       $rechts \leftarrow j$ 
13:    end if
14:  end for
15: end for
16: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

Zweiter Lösungsansatz

Eine Verbesserung ist es, nicht mehr jede Teilfolge explizit zu betrachten, sondern von jedem möglichen Startpunkt beginnend aufzusummieren.

Betrachtet man nun die Anzahl der Additionen in Zeile 6, so nimmt deren Anzahl nur noch in etwa proportional zu n^2 zu.

Algorithmen mit einem solchen Zeitverhalten nennt man **quadratische Algorithmen**.

MAXFOLGE2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:    $sum \leftarrow 0$ 
5:   for  $j \leftarrow i$  to  $n - 1$  do
6:      $sum \leftarrow sum + Z[j]$ 
7:     if  $sum > max$  then
8:        $max \leftarrow sum$ 
9:        $links \leftarrow i$ 
10:       $rechts \leftarrow j$ 
11:    end if
12:  end for
13: end for
14: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

Dritter Lösungsansatz „Teile-und-Herrsche“

Folgende Überlegung bringt uns zu einer noch besseren Zeitkomplexität:

Man teilt das Feld $Z[l \dots r]$ in der Mitte m , so dass zwei Teilabschnitte $Z[l \dots m]$ und $Z[m + 1 \dots r]$ entstehen. Der gesuchte Abschnitt mit der maximalen Summe kann sich nun

1. vollständig im Abschnitt $Z[l \dots m]$ oder
2. vollständig im Abschnitt $Z[m + 1 \dots r]$ befinden oder
3. sich über diese beiden Teilabschnitte erstrecken.

Maximale Abschnittssumme

FINDE-ZWISCHEN(Z, l, m, r)

```

1:  $linksMax \leftarrow -\infty$ 
2:  $sum \leftarrow 0$ 
3: for  $i \leftarrow m$  downto  $l$  do
4:    $sum \leftarrow sum + Z[i]$ 
5:   if  $sum > linksMax$  then
6:      $linksMax \leftarrow sum$ 
7:      $links \leftarrow i$ 
8:   end if
9: end for
10:  $rechtsMax \leftarrow -\infty$ 
11:  $sum \leftarrow 0$ 
12: for  $i \leftarrow m + 1$  to  $r$  do
13:    $sum \leftarrow sum + Z[i]$ 
14:   if  $sum > rechtsMax$  then
15:      $rechtsMax \leftarrow sum$ 
16:      $rechts \leftarrow i$ 
17:   end if
18: end for
19: return  $(linksMax + rechtsMax, links, rechts)$ 

```

Lösung für den 3. Fall

FINDE-ZWISCHEN behandelt 3. Fall, indem von m nach links und von $m + 1$ nach rechts die max. Abschnittssumme gesucht wird.

Es muss aus jedem Teilabschnitt min. ein Element enthalten sein, sonst wäre es nicht der 3. Fall!

Maximale Abschnittssumme

Dritter Lösungsansatz „Teile-und-Herrsche“

MAXFOLGE3(Z, l, r)

```

1: if  $l = r$  then                                ▷ Abbruchbedingung für die Rekursion
2:   return ( $Z[l], l, r$ )
3: end if
4:  $m \leftarrow (l + r) / 2$                             ▷ Ganzzahlige Division
5: ( $linksMax, linksL, linksR$ )  $\leftarrow$  MAXFOLGE3( $Z, l, m$ )    ▷ 1. Fall
6: ( $rechtsMax, rechtsL, rechtsR$ )  $\leftarrow$  MAXFOLGE3( $Z, m + 1, r$ )    ▷ 2. Fall
7: ( $zwiMax, zwiL, zwiR$ )  $\leftarrow$  FINDE-ZWISCHEN( $Z, l, m, r$ )    ▷ 3. Fall
8: if  $linksMax \geq rechtsMax$  and  $linksMax \geq zwiMax$  then
9:   return ( $linksMax, linksL, linksR$ )
10: end if
11: if  $rechtsMax \geq linksMax$  and  $rechtsMax \geq zwiMax$  then
12:   return ( $rechtsMax, rechtsL, rechtsR$ )
13: end if
14: return ( $zwiMax, zwiL, zwiR$ )

```

Es fällt auf, dass nur in der Funktion FINDE-ZWISCHEN Schleifen enthalten sind, in denen in Abhängigkeit der Größe von n Additionen durchgeführt werden – allerdings nicht verschachtelt.

Durch das wiederholte Halbieren des Feldes wird FINDE-ZWISCHEN etwa $\lg n$ - mal

aufgerufen, so dass die Anzahl der Additionen etwa proportional zu $n \lg n$

ist. ($\lg$ ist Logarithmus zur Basis 2)

Maximale Abschnittssumme

Vierter Lösungsansatz

Beim vierten Lösungsansatz wird die Folge nur einmal von links nach rechts durchlaufen und die Werte in *aktSum* aufsummiert.

Der erreichte maximale Summenwert wird in *max* gespeichert.

Wird *aktSum* kleiner als 0, so ist es besser, die Folge neu zu beginnen.

Die Anzahl der Additionen ist jetzt proportional zu *n*.

Solche Algorithmen nennt man **lineare Algorithmen**.

MAXFOLGE4(*Z*)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3:  $aktSum \leftarrow 0$ 
4:  $aktLinks \leftarrow 0$ 
5: for  $i \leftarrow 0$  to  $n - 1$  do
6:    $aktSum \leftarrow aktSum + Z[i]$ 
7:   if  $aktSum > max$  then
8:      $max \leftarrow aktSum$ 
9:      $links \leftarrow aktLinks$ 
10:     $rechts \leftarrow i$ 
11:   end if
12:   if  $aktSum < 0$  then
13:      $aktSum \leftarrow 0$ 
14:      $aktLinks \leftarrow i + 1$ 
15:   end if
16: end for
17: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

**Was bedeutet das bei einer
Zahlenfolge der Länge
 $n = 10000 = 10^4$?**

Die Anzahl der Additionen bei MAXFOLGE1 beträgt etwa $10^{4^3} = 1$ Billion

Die Anzahl der Additionen bei MAXFOLGE2 beträgt etwa 10^{4^2} (10000 mal schneller als MAXFOLGE1)

Die Anzahl der Additionen bei MAXFOLGE3 beträgt etwa $10^4 \cdot \lg 10^4 \approx 10^4 \cdot 13,3$ (751 mal schneller als MAXFOLGE2 und 7,5 Millionen mal schneller als MAXFOLGE 1)

Die Anzahl der Additionen bei MAXFOLGE4 beträgt etwa 10^4 (100 Millionen mal schneller als MAXFOLGE1)

**Test mit einer Rust-
Implementierung auf einem 3,4
GHz Intel Core i5**

7009 33 5140

MAXFOLGE1 (kubisch): 4738.40s

7009 33 5140

MAXFOLGE2 (quadratisch): 1.32s

7009 33 5140

MAXFOLGE3 (Teile-und-Herrsche): 4.85ms

7009 33 5140

MAXFOLGE4 (Linear): 302.49µs

Landau-Symbole

Landau-Symbole

Hinführung

Bei der Betrachtung der Komplexität versuchen wir, einen Term zu entwickeln, der die Anzahl der auszuführenden Anweisungen bzw. der benutzten Speicherzellen in Abhängigkeit von der Größe des Problems berechnet.

Beispiel

$$t(n) = 1,5n^2 + 0,5n - 1$$

Bei großen n ist nur der führende Term wichtig. Auch ignoriert man konstante Koeffizienten des führenden Terms. M.a.W.:

$$1,5n^2 + 0,5n - 1 \rightarrow \text{quadratische Komplexität}$$

Landau-Symbole

Historie

Das große O (großes Omikron) wurde Ende des 19. Jh von dem deutschen Zahlentheoretiker **Paul Bachmann** als Symbol für "Ordnung von" eingeführt.

Irrtümlicherweise wurde Notation nach dem ebenfalls deutschen Zahlentheoretiker **Edmund Landau** benannt, der diese Anfang des 20. Jh bekannt machte.

Mit den Landau-Symbole lassen sich Algorithmen hinsichtlich ihres (minimalen oder maximalen) Bedarfs an Zeit und/oder Speicher bezüglich eines bestimmten Maschinenmodells analysieren.

Man ermittelt dabei die Schranken, in denen sich Laufzeit und/oder Speicherplatzbedarf hält, wenn man die Problemgröße n vergrößert.

Landau-Symbole

Bedeutung

Notation	Bedeutung	Sprechweise
$f \in O(g)$	f wächst höchstens so schnell wie g	„groß <i>O</i> “
$f \in \Omega(g)$	f wächst mindestens so schnell wie g	„groß <i>Omega</i> “
$f \in \Theta(g)$	f wächst genauso schnell wie g	„groß <i>Theta</i> “

Mit den Landau-Symbolen beschreibt man eine Menge von Funktionen, deren Wachstumsraten vergleichbare Wachstumsraten haben wie die in Klammern angegebene Funktion.

Landau-Symbole

O-Notation

$O(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c \text{ und } n_0, \text{ so dass für alle } n \geq n_0 :$

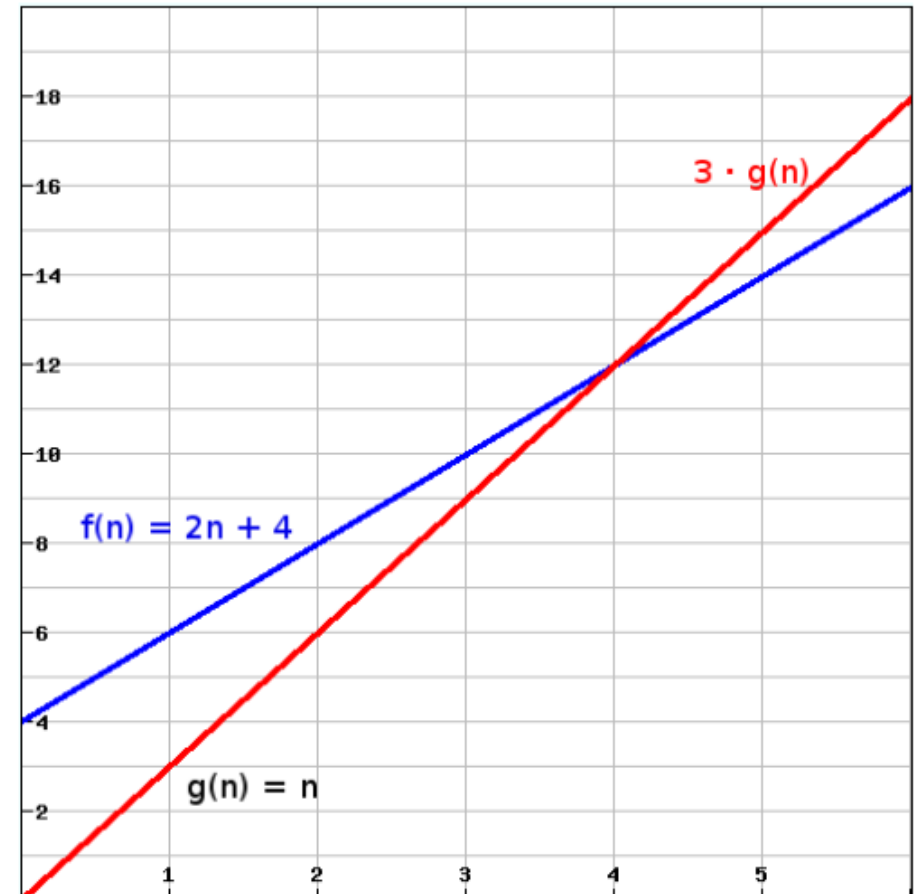
$$0 \leq f(n) \leq c \cdot g(n) \}$$

Beispiel

Gegeben $f(n) = 2n + 4$ und $g(n) = n$. Wir finden $c = 3$ und $n_0 = 4$, so dass

$$f(n) \leq 3 \cdot g(n) \text{ für alle } n \geq n_0$$

$$\rightarrow f(n) \in O(n)$$



Landau-Symbole

Θ -Notation

$$\Theta(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c_1, c_2 \text{ und } n_0, \text{ so dass für alle } n \geq n_0: \\ 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Man bezeichnet hier $g(n)$ auch als asymptotisch scharfe Schranke von $f(n)$, da f genauso schnell wie g wächst. Dies bedeutet, dass man die Wachstumsrate des Ressourcenbedarfs (bis auf konstante Faktoren) genau bestimmen kann.

Landau-Symbole

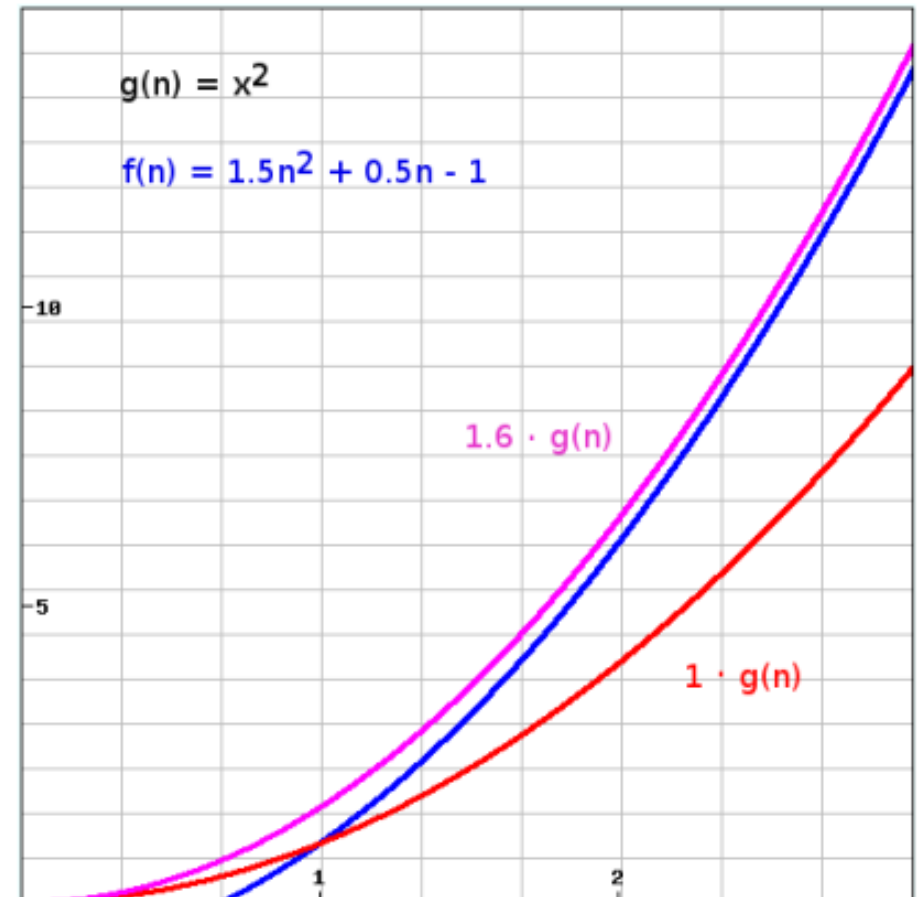
Θ -Notation

$\Theta(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c_1, c_2 \text{ und } n_0, \text{ so dass f\"ur alle } n \geq n_0 :$

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Beispiel

Gegeben $f(n) = 1,5n^2 + 0,5n - 1$ und $g(n) = n^2$. Wir finden $c_1 = 1$, $c_2 = 1,6$ und $n_0 = 1$



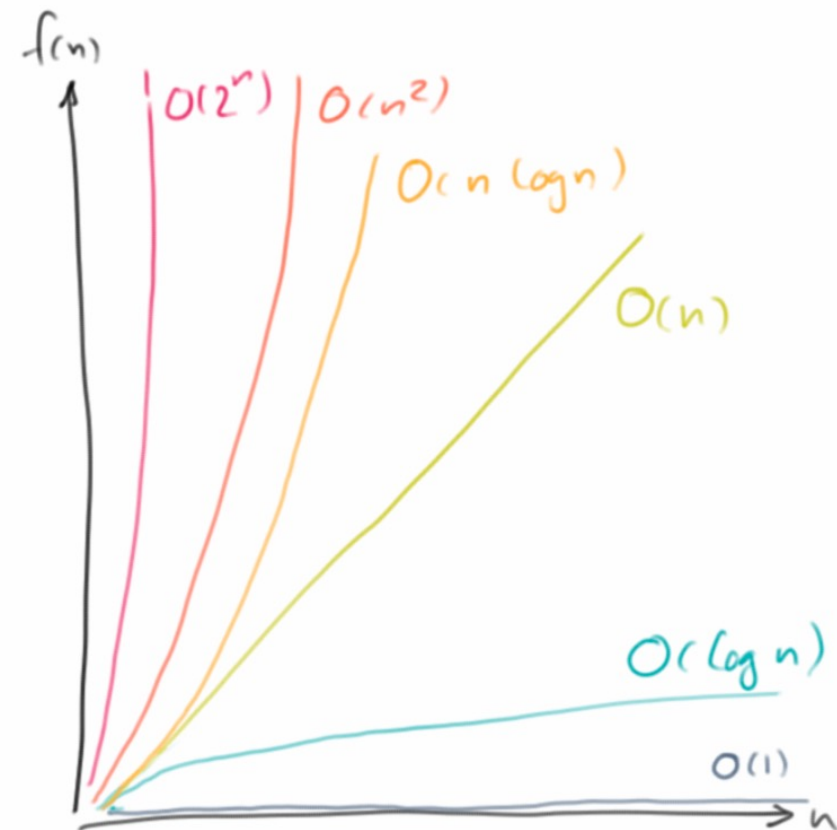
Landau-Symbole

Komplexitätsklasse

Eine **Komplexitätsklasse** fasst alle Algorithmen zusammen, deren Ressourcenbedarf dieselbe asymptotische Wachstumsrate hat.

Beispiel:

"MAXFOLGE4 hat lineare Komplexität"
bedeutet formal:
die Laufzeit von MAXFOLGE4 $\in \Theta(n)$.



Landau-Symbole

Ω -Notation

$$\Omega(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c \text{ und } n_0, \text{ so dass für alle } n \geq n_0: \\ 0 \leq c \cdot g(n) \leq f(n) \}$$

Die Ω -Notation gibt eine untere Schranke für die Wachstumsrate an.

$f(n) \in \Omega(g(n))$ bedeutet, dass der Ressourcenbedarf für alle $n \geq n_0$ mindestens proportional zu $g(n)$ wächst – unabhängig von der konkreten Eingabe.

In der Praxis wird Ω häufig genutzt, um die Best-Case-Laufzeit zu beschreiben.

Ω hm

Landau-Symbole

Beziehungen zwischen den Notationen

$$\Theta(f) \subseteq O(f)$$

$$\Theta(f) \subseteq \Omega(f)$$

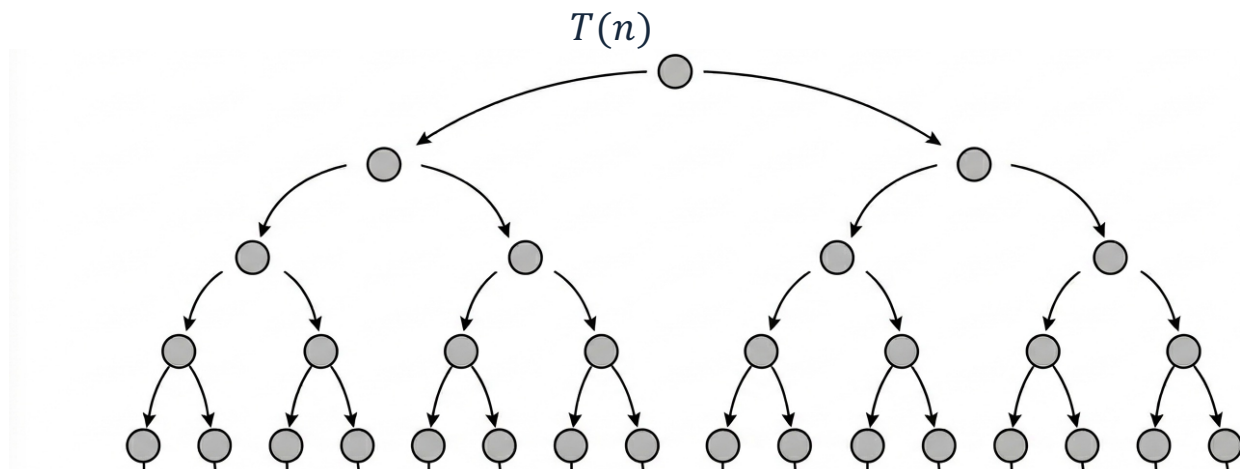
$$\Theta(f) = O(f) \cap \Omega(f)$$

Mastertheorem

Mastertheorem

Rekurrenz

Relativ einfach lässt sich meist die **Rekurrenz** (oder auch **Rekurrenzformel**) bestimmen, die den Ressourcenbedarf $T(n)$ auf den Bedarf für kleinere n zurückführt.



Mastertheorem

Rekurrenz bei MAXFOLGE3

Relativ einfach lässt sich meist die **Rekurrenz** (oder auch **Rekurrenzformel**) bestimmen, die den Ressourcenbedarf $T(n)$ auf den Bedarf für kleinere n zurückführt.

```

1: if  $l = r$  then                                ▷ Abbruchbedingung für die Rekursion
2:   return  $(Z[l], l, r)$ 
3: end if

```

Basisfall: $T(1) = 1$

```

5:  $(linksMax, linksL, linksR) \leftarrow \text{MAXFOLGE3}(Z, l, m)$            ▷ 1. Fall
6:  $(rechtsMax, rechtsL, rechtsR) \leftarrow \text{MAXFOLGE3}(Z, m + 1, r)$    ▷ 2. Fall
7:  $(zwiMax, zwiL, zwiR) \leftarrow \text{FINDE-ZWISCHEN}(Z, l, m, r)$        ▷ 3. Fall

```

Rekurrenz: $T(n) = 2 \cdot T(n/2) + n$

Mastertheorem

Allgemeine Rekurrenz

Die allgemeine Form $T(n) = a \cdot T(n/b) + f(n)$ kann zerlegt werden:

Parameter	Bedeutung	bei MAXFOLGE3
a	Anzahl der Teilprobleme	2
b	Verkleinerungsfaktor	2
$f(n)$	Kosten außerhalb der Rekursion	FINDE-ZWISCHEN $\in O(n)$

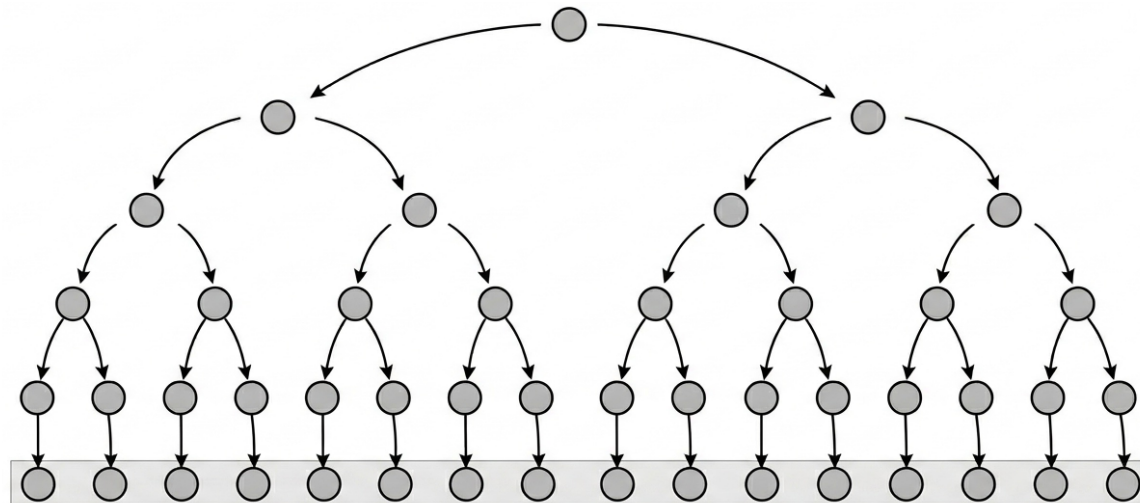
Mastertheorem

Zusammenhänge

Anzahl der Ebenen (Höhe des Baums): $\log_b n$; bei **MAXFOLGE3** ist $b = 2$, daher $\lg n$

Mit jeder Ebene entstehen a Teilprobleme, damit Anzahl der Blätter $a^{\log_b n} = n^{\log_b a}$

Bei **MAXFOLGE3**: $2^{\lg n} = n$



Mastertheorem

Grundidee

Der Aufwand für den rekursiven Algorithmus setzt sich aus zwei Komponenten zusammen:

1. An jedem "Blatt" des Rekursionsbaums fällt die Kosten des Basisfalls an.
2. Auf jeder Ebene des Rekursionsbaums fallen die Kosten außerhalb der Rekursion an (Teilen/Zusammenführen).

Die Gesamtkosten bzw. die Komplexitätsklasse hängt davon ab, welche Komponente dominiert.

Mastertheorem

Fallunterscheidung

1. Fall: Rekursion dominiert

$f(n)$ wächst langsamer als $n^{\log_b a} \rightarrow T(n) \in \Theta(n^{\log_b a})$

2. Fall: Gleichstand

$f(n)$ wächst genauso schnell $\rightarrow T(n) \in \Theta(f(n) \cdot \log_b n)$ (auf jeder Ebene $f(n)$)

3. Fall: Teilen/Zusammenführen dominiert

$f(n)$ wächst schneller $\rightarrow T(n) \in \Theta(f(n))$

Mastertheorem

Checkliste

1. Zuerst a , b und $f(n)$ identifizieren
2. Dann $n^{\log_b a}$ berechnen
3. $f(n)$ mit $n^{\log_b a}$ vergleichen
4. Passenden Fall anwenden

MAXFOLGE3

$$a = 2, b = 2 \text{ und } f(n) = n$$
$$n^{\log_b a} = n$$

Beide gleich $\rightarrow$ Fall 2

$$\Theta(f(n) \cdot \lg n)$$

Vielen Dank für die Aufmerksamkeit!